

מחברת 3: זיהוי תווים אופטי (OCR) לטקסטים עבריים

מבוא למדעי הרוח הדיגיטליים | אוניברסיטת אריאל | תשפ"ו

פרופ' שי גורדין

OCR (Optical Character Recognition) = זיהוי תווים אופטי – טכנולוגיה שהופכת תמונות של טקסט לטקסט ניתן לעריכה ולחיפוש. **נושאים:** (1) מהו OCR ולמה הוא חשוב? (2) Tesseract OCR – הכלי הפתוח המוביל (3) EasyOCR לעברית (4) הערכת איכות (5) OCR תיקון שגיאות (6) אתגרים מיוחדים בכתב יד עברי  **חשיבות לארכיוולוגיה ומדעי הרוח:** הגניזה הקהירית, כתבי יד ים המלח, תעודות עתיקות – כולם דורשים OCR לדיגיטציה.

מדריך שימוש במחברת

|||---|---| זמן מוערך | 🏠 בכיתה: ~50 דקות | 🏠 בבית: ~30 דקות || 🖥️ היכן להריץ | **Google Colab בלבד** – הפקודות `apt-get`! לא עובדות על מחשב רגיל || 🌐 **חיבור רשת** | נדרש להתקנות ולהורדת מודלים || ⚠️ **אזהרה** | EasyOCR מוריד מודל של ~2 ג'יגה-בייט – הפעלה ראשונה לוקחת 5–10 דקות |

! חשוב – הפקודות `apt-get`! עובדות רק ב-Colab

בניגוד לפקודות `!pip install`, הפקודה `apt-get install tesseract-ocr` מתקינה תוכנות מערכת. זה עובד ב-Colab כי Colab רץ על Linux בשרת Google. אם תנסו להריץ ב-Jupyter מקומי על Mac/Windows – הפקודה תיכשל. **השתמשו ב-Colab!**

סדר ריצה:

1. התקנת תוכנות (`!pip install + apt-get`)
2. יבוא ספריות
3. פונקציות עיבוד תמונה
4. יצירת תמונת דוגמה
5.  Tesseract OCR
6.  EasyOCR (אופציונלי – לוקח זמן)
7.  הערכת איכות
8.  תיקון שגיאות

חלק א: מהו OCR ולמה הוא חשוב?

OCR הוא תהליך שבו מחשב "קורא" טקסט מתמונה.

תהליך OCR:

תמונה ← עיבוד מוקדם ← פילוח שורות/תיבות ← זיהוי תווים ← תיקון שגיאות ← טקסט

דוגמאות לשימוש במדעי הרוח:

מקור	אתגר	פתרון
עיתונות היסטורית (מערב, הארץ)	דפוס ישן, פגמים	Tesseract + ניקוי
כתבי יד (גניזה קהירית)	כתב יד אישי	CNN + HTR
תעודות רשמיות עתיקות	קלף בלוי	שיפור תמונה + OCR
לוחות חרס (כתב יתד!)	צילום מיוחד	מודלים ייעודיים

מגבלות OCR לעברית:

- כיוון ימין-לשמאל (RTL) מסבך פילוח
- ניקוד (....) מסבך זיהוי
- אותיות דומות: ב/כ, ד/ר, ה/ח
- כתב יד מאוד שונה מדפוס לקריאה: [Springmann, U. et al. \(2018\). Ground Truth for Training OCR Engines on Historical Documents. DH2018](#)

```
In [ ]: # OCR התקנת ספריות
!pip install easyocr Pillow pytesseract -q
!apt-get install -y tesseract-ocr tesseract-ocr-heb > /dev/null 2>&1
print("✅ הותקנו OCR ספריות")
print("    • tesseract-ocr-heb - Tesseract בעברית")
print("    • easyocr - OCR מבוסס Deep Learning")
print("    • Pillow - עיבוד תמונות")
```

```
In [ ]: # יבוא ספריות
import pytesseract
from PIL import Image, ImageFilter, ImageEnhance, ImageOps
import easyocr
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import requests
from io import BytesIO
import os
import re
print("✅ ספריות יובאו")
```

חלק ב: עיבוד מוקדם של תמונות

למה עיבוד מוקדם חשוב?

תמונות "גולמיות" לרוב אינן אידיאליות ל-OCR. עיבוד מוקדם משפר דרמטית את הדיוק:

טכניקה	מטרה	מתי להשתמש
המרה לגווני אפור	הפחתת מורכבות	תמיד
ניגודיות	הבלטת הטקסט	טקסט דהוי
סף (Threshold)	שחור-לבן בינארי	רקע לא אחיד
הסרת רעש	חלקות	טקסט עם "כתמים"
ישר (Deskew)	תיקון זווית	תמונות סרוקות

```
In [ ]: # =====
# פונקציות עיבוד מוקדם של תמונות
# =====

def preprocess_image(image, method='standard'):
    """
    OCR. עיבוד מוקדם של תמונה לשיפור דיוק.

    פרמטרים:
        image : PIL Image
        method : 'standard' / 'aggressive' / 'gentle'

    מעובדת: PIL Image
    """
    # המרה לגווני אפור
    img = image.convert('L')

    if method == 'standard':
        # הגברת ניגודיות
        enhancer = ImageEnhance.Contrast(img)
        img = enhancer.enhance(2.0)
        # סף אדפטיבי
        img = img.point(lambda x: 0 if x < 128 else 255, '1')
        img = img.convert('L')

    elif method == 'aggressive':
        # חידוד קצוות
        img = img.filter(ImageFilter.SHARPEN)
        enhancer = ImageEnhance.Contrast(img)
        img = enhancer.enhance(3.0)
        img = img.point(lambda x: 0 if x < 100 else 255, '1')
        img = img.convert('L')

    elif method == 'gentle':
        # רק ניקוי קל
        img = img.filter(ImageFilter.MedianFilter(size=3))
```

```

        enhancer = ImageEnhance.Contrast(img)
        img = enhancer.enhance(1.5)

    return img

def show_preprocessing_comparison(image):
    """
    הצגת השוואה בין שיטות עיבוד מוקדם שונות.
    """
    methods = ['original', 'standard', 'aggressive', 'gentle']

    fig, axes = plt.subplots(1, 4, figsize=(16, 4))

    for i, method in enumerate(methods):
        if method == 'original':
            img_show = image.convert('L')
            title = 'מקורית'
        else:
            img_show = preprocess_image(image, method=method)
            title = method

        axes[i].imshow(img_show, cmap='gray')
        axes[i].set_title(f'שיטה: {title}', fontsize=12, fontweight='bold')
        axes[i].axis('off')

    plt.suptitle('השוואת שיטות עיבוד מוקדם', fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.savefig('01_preprocessing.png', dpi=150, bbox_inches='tight')
    plt.show()
    print(f"📁 01 ההשוואה_preprocessing.png")

print("✅ פונקציות עיבוד מוקדם מוגדרות")

```

```

In [ ]: # =====
# יצירת תמונת דוגמה עם טקסט עברי
# =====
# נצור תמונה עם טקסט עברי לדמו

from PIL import Image, ImageDraw, ImageFont
import numpy as np

def create_sample_hebrew_image(text_lines, width=600, height=200,
                               noise_level=10, font_size=24):
    """
    יצירת תמונת דוגמה עם טקסט עברי (לצורכי הדגמה).

    בדוגמה אמיתית, תשתמשו בתמונות סרוקות של מסמכים היסטוריים.
    """
    # יצירת תמונה לבנה
    img = Image.new('RGB', (width, height), color=(255, 255, 255))
    draw = ImageDraw.Draw(img)

    # כתיבת הטקסט (בלי פונט עברי מיוחד – נשתמש בברירת מחדל)
    y_pos = 20

```

```

for line in text_lines:
    draw.text((width - 20, y_pos), line, fill=(0, 0, 0), anchor='ra')
    y_pos += font_size + 8

# הוספת רעש קל (מדמה סריקה)
img_array = np.array(img)
noise = np.random.randint(-noise_level, noise_level, img_array.shape)
img_array = np.clip(img_array.astype(int) + noise, 0, 255).astype(np.uint8)

return Image.fromarray(img_array)

# דוגמאות לטקסטים עבריים
sample_texts = [
    ["חפירות ארכיאולוגיות בארץ ישראל",
     "מגלות שכבות היסטוריות רבות",
     "מהתקופה הכנענית ועד העות'מאנית"],

    ["הגניזה הקהירית",
     "אוסף של כ-300,000 קטעי מסמכים",
     "שנמצאו בבית הכנסת בן-עזרא, קהיר"],

    ["מדעי הרוח הדיגיטליים",
     "משלבים כלים חישוביים",
     "עם מחקר הומניסטי"]
]

# יצירת תמונות דוגמה
sample_images = []
for i, lines in enumerate(sample_texts):
    img = create_sample_hebrew_image(lines, noise_level=15)
    sample_images.append(img)
    img.save(f'sample_hebrew_{i+1}.png')
    print(f"✅ נוצרה תמונת דוגמה {i+1}")

# הצגת התמונות
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
titles = ['גניזה קהירית', 'ארכיאולוגיה', 'DH']
for i, (img, title) in enumerate(zip(sample_images, titles)):
    axes[i].imshow(img)
    axes[i].set_title(title, fontsize=12, fontweight='bold')
    axes[i].axis('off')
plt.suptitle('OCR תמונות דוגמה ל', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
print("\n💡 תשתמשו בתמונות סרוקות של מסמכים היסטוריים")

```

חלק ג: Tesseract OCR

מהו Tesseract?

Tesseract הוא מנוע OCR קוד-פתוח פותח על ידי Google, ונחשב לאחד הטובים בעולם למסמכים מודפסים. יתרונות: חינמי, תומך ב-100+ שפות כולל עברית, קל לשימוש

הפרמטרים החשובים:

- `lang='heb'` – שפה עברית
- `config='--psm 6'` – מצב פילוח (6 = בלוק טקסט אחד)
- PSM modes: 3=automatic, 6=single block, 11=single word per line

```
In [ ]: # =====
# Tesseract OCR לעברית
# =====

def run_tesseract(image, lang='heb', psm=6):
    """
    על תמונה Tesseract OCR הרצת.

    פרמטרים:
        image : PIL Image
        lang   : שפה ('heb', 'eng', 'heb+eng')
        psm    : Page Segmentation Mode (3=אחד בלוק, 6=אוטומטי)

    מחזיר: הטקסט שזוהה
    """
    config = f'--psm {psm} --oem 3'
    text = pytesseract.image_to_string(
        image,
        lang=lang,
        config=config
    )
    return text.strip()

def tesseract_with_details(image, lang='heb'):
    """
    עם מידע מפורט על כל תיבת זיהוי Tesseract.
    עם מיקום ורמת ביטחון לכל מילה DataFrame מחזיר.
    """
    import pandas as pd
    data = pytesseract.image_to_data(
        image, lang=lang,
        output_type=pytesseract.Output.DATAFRAME
    )
    # סינון תוצאות עם ביטחון מספיק
    data = data[data['conf'] > 0].copy()
    return data

# — על הדוגמאות Tesseract הרצת —
print("🔍 על תמונות הדוגמה Tesseract OCR מריץ")
print("=" * 55)

for i, img in enumerate(sample_images[:2]): # ראשונות 2
    print(f"\n🖼️ תמונה {i+1}:")
    print("-" * 40)
```

```
# ניסיון עם תמונה מקורית
text_raw = run_tesseract(img, lang='heb')
print(f"מקורית: {repr(text_raw[:100])}")

# ניסיון עם עיבוד מוקדם
img_processed = preprocess_image(img, method='standard')
text_processed = run_tesseract(img_processed, lang='heb')
print(f"מעובדת: {repr(text_processed[:100])}")

print("\n💡 שים לב: התמונות נוצרו ללא פונט עברי ייעודי – לכן
print("בפרויקט אמיתי עם תמונות סרוקות, הדיוק גבוה הרבה יותר")
```

חלק ד: OCR – EasyOCR מבוסס Deep Learning

מהו EasyOCR?

EasyOCR הוא כלי OCR מודרני המבוסס על רשתות נוירונים עמוקות (Deep Learning). יתרונות לעברית:

- עדיף על Tesseract לכתב יד
- מטפל טוב יותר בפונטים מגוונים
- מחזיר רמת ביטחון (confidence) לכל מילה
- אין צורך בעיבוד מוקדם מורכב **חסרונות**: איטי יותר, דורש GPU לביצועים מיטביים

```
In [ ]: # =====
# EasyOCR? הגדרה: האם להפעיל
# =====
# מוריד מודל של ~2 ג'יגה-בייט בהפעלה הראשונה.
# 10 Mbps בחיבור אינטרנט של 25 דקות –
#
# בכיתה – נריץ יחד True השאירו.
# אם False-שנו ל:
# החיבור איטי –
# רוצים לדלג ולהגיע לתרגילים –

SKIP_EASYOCR = False # ← True-שנו ל

if SKIP_EASYOCR:
    print("🟡 EasyOCR מושבת – נדלג לחלק הערכת האיכות")
else:
    print("🟢 EasyOCR יופעל – ההורדה הראשונה עשויה לקחת כמה דקות")
    print("(הראשון שלכם, המודל כבר שמור בקאש Colab-אם זה ה)")
```

```
In [ ]: # =====
# EasyOCR – OCR מבוסס Deep Learning
# =====
# ⌚ ~2 המתינו בסבלנות – GB ההפעלה הראשונה מורידה

if SKIP_EASYOCR:
    print("🟡 EasyOCR מושבת – דלגנו")
    print("בתא למעלה SKIP_EASYOCR=False כדי להפעיל, שנו")
```

```

reader = None # placeholder
else:
    print("🤖 מאתחל EasyOCR...")
    print("(עשוי לקחת מספר דקות GB, הורדה ראשונה = כ-2)")
    print("חדש המודל מוריד כל פעם - זה נורמלי Colab בסטן")
    reader = easyocr.Reader(['he', 'en'], gpu=False)
    print("✅ EasyOCR מוכן!")

def run_easyocr(image_path_or_array, reader):
    """
    על תמונה EasyOCR הרצת.
    tuple (bounding_box, text, confidence) מחזיר: רשימת
    """
    if reader is None:
        print("⚠️ EasyOCR שנו SKIP_EASYOCR=False מושבת. שנו")
        return []
    results = reader.readtext(image_path_or_array)
    return results

def display_easyocr_results(image, results, title='EasyOCR'):
    """
    על התמונה bounding boxes עם EasyOCR הצגת תוצאות
    """
    import numpy as np
    img_arr = np.array(image)
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    # bounding boxes ציור
    img_with_boxes = img_arr.copy()
    for (bbox, text, conf) in results:
        pts = [(int(p[0]), int(p[1])) for p in bbox]
        # ציור מלבן
        import cv2 # noqa
        cv2.rectangle(img_with_boxes, pts[0], pts[2], (0, 255, 0), 2)

    ax1.imshow(img_arr)
    ax1.set_title('תמונה מקורית', fontsize=14)
    ax1.axis('off')

    ax2.imshow(img_with_boxes)
    ax2.set_title(f'{title} - תוצאות', fontsize=14)
    ax2.axis('off')

    plt.suptitle(title, fontsize=16, fontweight='bold')
    plt.tight_layout()
    plt.show()

    print("\n📝 טקסט שזוהה:")
    for i, (bbox, text, conf) in enumerate(results):
        print(f" {i+1}. {text:30s} ביטחון: {conf:.1%}")

```

חלק ה: הערכת איכות OCR

מדד	הסבר	נוסחה
CER (Character Error Rate)	שיעור שגיאות ברמת תווים	(הכנסות + מחיקות + החלפות) / סה"כ תווים
WER (Word Error Rate)	שיעור שגיאות ברמת מילים	(שגיאות מילים) / סה"כ מילים

:Ground Truth

כדי לחשב CER/WER, צריך **Ground Truth** – הטקסט הנכון לידנו.
בפרויקטי OCR, יוצרים Ground Truth על ידי הקלדה ידנית של חלק מהמסמכים.

```
In [ ]: # =====
# OCR: CER ו-WER חישוב מדדי איכות
# =====

def levenshtein_distance(s1, s2):
    """
    חישוב מרחק לוונשטיין (מספר פעולות עריכה מינימלי בין שני מחרוזות).
    CER ו-WER בסיס לחישוב
    """
    if len(s1) < len(s2):
        return levenshtein_distance(s2, s1)

    if len(s2) == 0:
        return len(s1)

    prev_row = range(len(s2) + 1)

    for i, c1 in enumerate(s1):
        curr_row = [i + 1]
        for j, c2 in enumerate(s2):
            insertions = prev_row[j + 1] + 1
            deletions = curr_row[j] + 1
            substitutions = prev_row[j] + (c1 != c2)
            curr_row.append(min(insertions, deletions, substitutions))
        prev_row = curr_row

    return prev_row[-1]

def compute_cer(ground_truth, ocr_output):
    """
    CER – Character Error Rate.
    מושלם, 100% = 1 שגיאות = 0
    """
    gt_clean = re.sub(r'\s+', ' ', ground_truth.strip())
    ocr_clean = re.sub(r'\s+', ' ', ocr_output.strip())

    if len(gt_clean) == 0:
        return 0.0

    dist = levenshtein_distance(gt_clean, ocr_clean)
```

```

return dist / len(gt_clean)

def compute_wer(ground_truth, ocr_output):
    """
    חישוב WER – Word Error Rate.
    """
    gt_words = ground_truth.strip().split()
    ocr_words = ocr_output.strip().split()

    if len(gt_words) == 0:
        return 0.0

    dist = levenshtein_distance(gt_words, ocr_words)
    return dist / len(gt_words)

# — הדגמה —
print("📊 חישוב CER/WER:")
print("=" * 55)

# דוגמאות השוואה
examples = [
    ("פפירות ארכאולוגיות", "פפירות ארכיאולוגיות"), # שגיאת כתיב קטנה
    ("הגניזה קהירית", "הגניזה הקהירית"), # מילה חסרה
    ("מדעי הרוח הדיגיטליים", "מדעי הרוח הדיגיטליים"), # שגיאת טיפוסית OCR
]

for gt, ocr in examples:
    cer = compute_cer(gt, ocr)
    wer = compute_wer(gt, ocr)
    print(f"\n GT : {gt}")
    print(f" OCR: {ocr}")
    print(f" CER={cer:.3f} ({cer*100:.1f}%) | WER={wer:.3f} ({wer*100:.1f}%)")

print("\n💡 CER < 5% = OCR איכותי מאוד")
print(" CER < 15% = OCR (ניתן לתיקון)")
print(" CER > 30% = OCR (דורש שיפור)")

```

חלק ו: תיקון שגיאות OCR

שיטות תיקון:

1. תיקון ידני: הכי מדויק אבל איטי
2. חיפוש במילון: בדיקת כל מילה מול מילון עברי
3. מודלי שפה: GPT/Claude לתיקון הקשרי
4. כלים ייעודיים: Transkribus, eScriptorium

Transkribus – כלי מוביל לכתב יד:

Transkribus מיועד לכתבי יד היסטוריים ומשתמש בלמידה עמוקה. מחקרי גניזה קהירית משתמשים בו נרחבות.

```

In [ ]: # =====
# תיקון שגיאות OCR
# =====

# מילון תיקונים נפוצים לעברית (OCR errors)
COMMON_OCR_CORRECTIONS = {
    # אותיות דומות
    '0': 'ה', 'ה' + '0': 'ה',
    '1': 'ר', 'ר' + '1': 'ר',
    '0': 'ו', 'ו' + '0': 'ו',
    '1': 'י', 'י' + '1': 'י',

    # מילים נפוצות שנגרסות
    '1': 'ישראל',
    '1': 'ירושלים',
    '0': 'ארכאולוגיה',

}

def basic_ocr_correction(text, corrections=None):
    """
    נפוצות OCR תיקון בסיסי של שגיאות.

    פרמטרים:
        text : טקסט פלט
        corrections : {שגוי: נכון} מילון תיקונים

    מחזיר: טקסט מתוקן
    """
    if corrections is None:
        corrections = COMMON_OCR_CORRECTIONS

    corrected = text
    for wrong, right in corrections.items():
        corrected = corrected.replace(wrong, right)

    return corrected

def spell_check_hebrew(text, word_list=None):
    """
    בדיקת איות בסיסית לעברית.
    משווה מול רשימת מילים נפוצות.

    hunspell-he-הערה: לתיקון מקצועי, מומלץ להשתמש ב-
    """
    if word_list is None:
        # מדגם של מילים נפוצות בתחום
        word_list = set([
            'ארכאולוגיה', 'חפירה', 'מצאים', 'שכבה', 'אתר',
            'היסטוריה', 'תקופה', 'ממלכה', 'שושלת', 'מלחמה',
            'דיגיטלי', 'מחשב', 'נתונים', 'אלגוריתם', 'קורפוס',
            'ירושלים', 'תל', 'חרס', 'ברונזה', 'ברזל',
            'ישראל', 'יהודה', 'שומרון', 'גליל', 'נגב'
        ])

```

```

words = text.split()
unknown = []
for word in words:
    clean = re.sub(r'^\u05D0-\u05EA', '', word)
    if len(clean) > 2 and clean not in word_list:
        unknown.append(clean)

return unknown

# — הדגמה —
print("🔧 הדגמת תיקון שגיאות OCR:")
print("=" * 55)

test_texts = [
    "הא נמצא בחפירות ב10 שלם",
    "ממצא 1 ארכאולוגים חשובים",
    "מדע 1 הרחב הדגל1 טל1ים"
]

for text in test_texts:
    corrected = basic_ocr_correction(text)
    print(f"\n שגוי: {text}")
    print(f" מתוקן: {corrected}")

print("\n💡 לתיקון מקצועי: ")
print(" pip install pyenchant # בדיקת איות ")
print(" Claude API-או שימוש ב לתיקון הקשרי ")

```

חלק ז: Pipeline מלא – מתמונה לטקסט ניתן לחיפוש

תהליך מלא לדיגיטציה:

1. סריקה → DPI +300, אפור/צבע
2. עיבוד מוקדם → ניגודיות, ישור, חיתוך
3. OCR → Tesseract / EasyOCR
4. הערכה → CER/WER מול Ground Truth
5. תיקון → ידני / אוטומטי
6. אינדקס → חיפוש פולטקסט

```

In [ ]: # =====
# Pipeline - מתמונה לטקסט
# =====

def full_ocr_pipeline(image, lang='heb', method='standard'):
    """
    לעברית OCR מלא של Pipeline.

    שלבים:
    1. עיבוד מוקדם
    2. Tesseract OCR
    """

```

3. תיקון בסיסי

```
עם כל שלבי העיבוד dict מחזיר:
"""
results = {
    'original_image': image,
    'steps': {}
}

# שלב 1: עיבוד מוקדם
preprocessed = preprocess_image(image, method=method)
results['steps']['preprocessing'] = preprocessed

# שלב 2: OCR שיטות שונות
text_raw = run_tesseract(image, lang=lang)
text_processed = run_tesseract(preprocessed, lang=lang)
results['steps']['tesseract_raw'] = text_raw
results['steps']['tesseract_processed'] = text_processed

# שלב 3: תיקון
text_corrected = basic_ocr_correction(text_processed)
results['steps']['corrected'] = text_corrected

# שלב 4: מידע נוסף
results['word_count'] = len(text_corrected.split())
results['char_count'] = len(re.sub(r'\s', '', text_corrected))

return results

# — Pipeline-הרצת ה —
print("🔄 מריץ Pipeline OCR מלא...")
print("=" * 55)

for i, img in enumerate(sample_images):
    print(f"\n📄 מסמך {i+1}:")
    result = full_ocr_pipeline(img)

    print(f"  Tesseract (גולמי) : {repr(result['steps']['tesseract_raw'][:60])}")
    print(f"  Tesseract (מעובד) : {repr(result['steps']['tesseract_processed'][:60])}")
    print(f"  לאחר תיקון : {repr(result['steps']['corrected'][:60])}")
    print(f"  מילים: {result['word_count']} | תווים: {result['char_count']}")

print("\n✅ Pipeline הושלם!")
```

תרגילים 

★ תרגיל 1 – בסיסי

הורידו תמונה של עיתון עברי ישן מ[National Library of Israel](https://www.nli.org.il/en/subject-bases/ocr) והריצו עליה OCR.

השוו: Tesseract לעומת EasyOCR – מי מדויק יותר?

★★ תרגיל 2 – בינוני

צרו **Ground Truth** קטן (5-10 שורות) והשוו את דיוק Tesseract לפני ואחרי עיבוד מוקדם.
חשבו CER ו-WER לכל שיטה.

תרגיל 3 – מתקדם ★★☆☆

השתמשו ב-Claude API לתיקון שגיאות OCR:

```
# עם הנחיה Claude ל-OCR שלחו את טקסט ה
# :בטקסט העברי הבא, מבלי לשנות תוכן OCR-תקן את שגיאות ה"
```

משאבים 🔗

- [Tesseract OCR Documentation](#)
- [EasyOCR GitHub](#)
- [Transkribus](#) – לכתב יד היסטורי
- [National Library of Israel – OCR](#)
- [Programming Historian: OCR with Tesseract](#)

מטלת בית – OCR על טקסט אמיתי 🏠

מה לעשות:

1. בחרו **טקסט מודפס עברי** – עיתון ישן, ספר, או מסמך (שני עמודים לפחות)
2. **סרקו** באמצעות הסורק או אפליקציית הטלפון (Microsoft Lens / Google PhotoScan)
3. **העלו ל-Colab** את קובץ התמונה (גרירה לחלונית הקבצים בצד שמאל)
4. הריצו OCR עם `run_tesseract()` על התמונה שלכם
5. **בדקו** 10 שורות ידנית וסמנו שגיאות בצבע אחר

העלאת קובץ ל-Colab:

```
# לאחר העלאה, קראו את התמונה:
from PIL import Image
my_image = Image.open('שם_הקובץ.jpg') # שנו לשם הקובץ שלכם
my_text = run_tesseract(my_image, lang='heb')
print(my_text)
```

מה להגיש:

- קובץ `OCR_output.txt` עם הטקסט שזוהה
- דוח קצר (טבלה): שורות שנבדקו, שגיאות שנמצאו, CER מוערך

צריכים עזרה? שאלו את Gemini:

"אני מנסה להריץ Tesseract OCR בפייתון על תמונה עברית. קיבלתי: [שגיאה]. כיצד לפתור?"

רשימת בדיקה לפני הגשה

- ☐ הרצתי OCR על תמונה אמיתית (לא על תמונת הדוגמה מהמחברת)
- ☐ בדקתי לפחות 10 שורות ידנית
- ☐ יש לי קובץ טקסט פלט
- ☐ כתבתי הערות על שגיאות שמצאתי